

DeUna Nearby: Implementación de un Motor de Recomendaciones Contextuales para Comercios Cercanos en una Aplicación Fintech

Documento técnico del proyecto
DevIAthon - Categoría DeUna

Autor: Anthony López

, Sebastián Eriquez, Jhon Guerron, Taisha Yar

Fecha: Mayo de 2026

Institución/Equipo: Equipo DeUna Nearby



Resumen

DeUna Nearby es un módulo propuesto para integrarse dentro de la aplicación DeUna con el propósito de transformar una experiencia principalmente transaccional en una experiencia de descubrimiento comercial inteligente. La solución permite recomendar negocios cercanos de forma personalizada mediante el análisis del historial de compras del usuario, su ubicación aproximada, las promociones activas de los comercios afiliados y un score de afinidad calculado por un motor de recomendación contextual. El proyecto se desarrolló como un Producto Mínimo Viable (MVP) para un entorno de hackathon, usando Next.js, TypeScript, Supabase y PostgreSQL. El backend incorpora APIs REST, una función RPC en Supabase, cálculo de distancia mediante coordenadas y endpoints listos para que el frontend, el módulo de notificaciones y el dashboard consuman la información. El resultado es una arquitectura funcional, escalable y orientada a demostrar valor para tres actores: usuarios, comercios y DeUna.

Palabras clave: fintech, recomendaciones, Supabase, proximidad, comercios locales, MVP, DeUna.

Tabla de Contenido

1. Introducción
 2. Planteamiento del problema
 3. Justificación
 4. Objetivos
 5. Alcance del MVP
 6. Marco conceptual y tecnológico
 7. Metodología de desarrollo
 8. Arquitectura del sistema
 9. Diseño de base de datos
 10. Motor de recomendación contextual
 11. Proximidad geográfica y cálculo de distancia
 12. APIs implementadas
 13. Integración con frontend y roles del equipo
 14. Seguridad, variables de entorno y buenas prácticas
 15. Pruebas y validación
 16. Resultados esperados e impacto
 17. Limitaciones y trabajo futuro
 18. Conclusiones
- Referencias
- Apéndices

1. Introducción

Las aplicaciones fintech han evolucionado desde herramientas enfocadas únicamente en pagos hacia ecosistemas digitales capaces de generar valor adicional para usuarios y comercios. En este contexto, DeUna Nearby se plantea como una funcionalidad integrada dentro de la aplicación DeUna para recomendar comercios cercanos y beneficios personalizados. La propuesta no busca crear una aplicación independiente, sino añadir una capa inteligente dentro de la experiencia existente de DeUna, manteniendo la lógica de simplicidad, rapidez y utilidad que caracteriza a las aplicaciones móviles de pagos.

El proyecto parte de una necesidad clara: muchos usuarios abren aplicaciones financieras solo cuando necesitan pagar, transferir o consultar un saldo. Esto reduce la frecuencia de uso y limita las oportunidades de monetización. Por otro lado, los pequeños y medianos comercios suelen tener poca visibilidad frente a cadenas grandes o plataformas de publicidad con mayor presupuesto. DeUna Nearby conecta ambos problemas mediante recomendaciones basadas en datos: el usuario recibe sugerencias relevantes y el comercio obtiene exposición ante clientes con alta probabilidad de compra.

El documento técnico describe la arquitectura, el diseño de datos, la lógica de recomendación, los endpoints implementados, el uso de Supabase, la integración con el frontend y las decisiones técnicas tomadas durante el desarrollo del MVP. Además, se explica cómo defender la solución ante un jurado técnico, indicando qué parte corresponde a inteligencia artificial aplicada y cómo la proximidad puede evolucionar hacia GPS real en una versión de producción.

2. Planteamiento del Problema

El problema central consiste en que una parte importante de los usuarios de una billetera digital puede permanecer pasiva o utilizar la aplicación únicamente para transacciones puntuales. Esta situación reduce el potencial de generación de ingresos, limita la interacción con la aplicación y desaprovecha la información transaccional que podría utilizarse para crear experiencias personalizadas.

Simultáneamente, los comercios pequeños enfrentan dificultades para adquirir nuevos clientes. Aunque puedan estar afiliados a un ecosistema de pagos, su visibilidad puede ser baja si el usuario no tiene una razón concreta para descubrirlos. Las promociones genéricas no siempre son efectivas, porque no consideran el contexto del usuario ni sus hábitos de compra. Por ello, se requiere una funcionalidad que combine intención de consumo, cercanía y beneficios en tiempo real.

Actor	Problema observado	Consecuencia
Usuario	No recibe recomendaciones relevantes dentro de la app	Abre la aplicación solo cuando necesita pagar
Comercio local	Tiene baja visibilidad frente a usuarios cercanos	Pierde oportunidades de venta
DeUna	No todos los usuarios generan transacciones frecuentes	Se reduce el potencial de monetización y fidelización

3. Justificación

La implementación de DeUna Nearby se justifica porque aprovecha datos que una plataforma fintech ya puede manejar de manera estructurada: historial de transacciones, categorías de consumo, ubicación aproximada, comercios afiliados y promociones activas. En lugar de mostrar publicidad general, el sistema entrega recomendaciones con base en señales de comportamiento y contexto. Esto mejora la percepción del usuario, ya que la promoción se presenta como una oportunidad útil y no como un anuncio invasivo.

Desde el punto de vista comercial, la solución permite que los negocios afiliados reciban tráfico más calificado. No se trata solo de aparecer en una lista, sino de aparecer frente a usuarios

que tienen afinidad con su categoría y que se encuentran cerca. Para DeUna, el beneficio está en aumentar aperturas de la aplicación, transacciones, retención de usuarios y posibles modelos de monetización mediante promociones destacadas, campañas inteligentes o planes para comercios.

4. Objetivos

4.1 Objetivo General

Diseñar e implementar un MVP de DeUna Nearby como una funcionalidad integrada a la aplicación DeUna que recomiende comercios cercanos de manera personalizada mediante historial de compras, promociones activas y proximidad geográfica.

4.2 Objetivos Específicos

- Construir un backend funcional con Next.js y TypeScript para exponer APIs consumibles por el frontend.
- Configurar Supabase y PostgreSQL como base de datos para comercios, promociones e historial de transacciones.
- Implementar un motor de recomendación contextual capaz de calcular un score de afinidad.
- Incorporar cálculo de proximidad mediante coordenadas de latitud y longitud.
- Crear endpoints que permitan a los módulos de home, notificaciones y dashboard consumir información real o simulada de forma estructurada.
- Documentar la arquitectura para facilitar la integración del equipo y la defensa del proyecto ante el jurado.

5. Alcance del MVP

El Producto Mínimo Viable del proyecto se diseñó para demostrar el valor central de la solución sin construir una aplicación de producción completa. El MVP valida que, al combinar

historial de compras, proximidad y promociones, se pueden generar recomendaciones relevantes dentro de la experiencia móvil de DeUna.

Incluido en el MVP	No incluido en el MVP
Backend con APIs Next.js	Sistema real de pagos de DeUna
Supabase con tablas y RPC	Integración oficial con infraestructura interna de DeUna
Recomendación contextual	Modelo de machine learning entrenado con grandes volúmenes de datos
Proximidad por coordenadas	GPS real en app nativa publicado en producción
Notificación inteligente simulada	Push notifications reales mediante FCM/APNs
Dashboard/pitch con métricas simuladas	Analítica empresarial completa en tiempo real

Esta delimitación permite entregar una demo funcional en poco tiempo y, al mismo tiempo, dejar clara la ruta técnica hacia una versión más robusta. En un hackathon, el objetivo no es completar una plataforma empresarial definitiva, sino demostrar con claridad que la idea funciona, tiene impacto y puede escalar.

6. Marco Conceptual y Tecnológico

6.1 Aplicaciones Fintech y Descubrimiento Comercial

Una aplicación fintech puede ampliar su valor si deja de ser únicamente un medio de pago y se convierte en un punto de interacción comercial. En este proyecto, la recomendación de negocios cercanos convierte a DeUna en un canal de descubrimiento: el usuario no solo paga, también encuentra beneficios y comercios relevantes.

6.2 Sistemas de Recomendación

Un sistema de recomendación es un mecanismo que prioriza opciones para un usuario según señales disponibles. En este caso, el sistema no se basa en un modelo complejo entrenado, sino en un motor contextual con reglas ponderadas. Esto es apropiado para un MVP porque permite explicar fácilmente el resultado y ajustar el score sin depender de grandes datasets.

6.3 Next.js y Route Handlers

Next.js permite construir interfaces y endpoints dentro de una misma aplicación. En el App Router, los Route Handlers permiten crear manejadores personalizados para solicitudes HTTP dentro del directorio app. Esta característica se utilizó para implementar endpoints como /api/recommendations y /api/nearby-businesses, que entregan datos al frontend y a los módulos del equipo (Next.js, 2026).

6.4 Supabase y PostgreSQL

Supabase se utilizó como backend de datos porque proporciona una base PostgreSQL administrada y una librería JavaScript que permite interactuar con la base de datos desde aplicaciones web. El cliente de Supabase requiere una URL del proyecto y una clave pública o publishable key para inicializar la conexión (Supabase, 2026).

6.5 Funciones RPC en PostgreSQL

PostgreSQL permite crear funciones en la base de datos mediante CREATE FUNCTION. En el proyecto, se utilizó una función RPC llamada get_nearby_merchants_with_user_stats para devolver comercios desde Supabase y permitir que el endpoint indique source: supabase cuando los datos provienen de la base real (PostgreSQL Global Development Group, 2026).

7. Metodología de Desarrollo

El desarrollo se organizó por módulos para permitir trabajo paralelo. El equipo se dividió en cuatro roles: backend, frontend home, geolocalización/notificaciones y dashboard/pitch. El rol backend se encargó de Supabase, APIs, motor de recomendación y proximidad. El resto del equipo consumió las APIs para construir la experiencia visible.

1. Definición del problema y propuesta de valor.
2. Diseño de arquitectura mínima para MVP.
3. Configuración de repositorio GitHub y ramas por integrante.

4. Creación de base de datos en Supabase y datos de prueba.
5. Implementación de APIs en Next.js.
6. Implementación del motor de recomendación y cálculo de proximidad.
7. Integración con frontend, notificaciones y dashboard.
8. Pruebas locales y corrección de rutas/dependencias.

La metodología se basó en iteraciones cortas. Primero se logró una conexión básica a Supabase, luego se agregaron endpoints, después se incorporó proximidad y finalmente se preparó la integración para los compañeros. Esta estrategia permitió avanzar sin bloquear al equipo completo.

8. Arquitectura del Sistema

La arquitectura de DeUna Nearby se diseñó para separar responsabilidades. El frontend no calcula toda la lógica de negocio, sino que consume APIs. El backend consulta Supabase, calcula scores y devuelve respuestas listas para usar en componentes móviles.

```
Frontend móvil DeUna
-> APIs Next.js
-> Motor de recomendación contextual
-> Supabase / PostgreSQL
-> merchants, promotions, transactions
```

Capa	Responsabilidad	Implementación
Presentación	Mostrar home, cards, notificaciones y dashboard	Next.js frontend y componentes móviles
API	Entregar datos estructurados a cada módulo	Route Handlers de Next.js
Lógica	Calcular score, distancia y mensajes	Módulos en src/lib
Datos	Guardar comercios, promociones e historial	Supabase/PostgreSQL
Integración	Permitir trabajo de varios compañeros	GitHub por ramas

9. Diseño de Base de Datos

La base de datos se diseñó con tres entidades principales: merchants, promotions y transactions. Estas tablas permiten representar los comercios disponibles, sus beneficios y el comportamiento transaccional del usuario.

9.1 Tabla merchants

La tabla merchants almacena información de los comercios. Incluye coordenadas para calcular proximidad. En producción, estos registros corresponderían a comercios afiliados a DeUna.

Campo	Tipo	Descripción
id	uuid	Identificador único del comercio
name	text	Nombre comercial
category	text	Categoría del comercio
description	text	Descripción corta
latitude	double precision	Latitud del comercio
longitude	double precision	Longitud del comercio
active	boolean	Indica si el comercio está activo

9.2 Tabla promotions

La tabla promotions almacena beneficios vinculados a comercios. Esta tabla permite separar la información comercial del descuento disponible, facilitando cambios futuros en campañas.

Campo	Tipo	Descripción
id	uuid	Identificador único de la promoción
merchant_id	uuid	Relación con el comercio
title	text	Nombre de la promoción
description	text	Detalle del beneficio
discount	integer	Porcentaje de descuento o beneficio
active	boolean	Estado de la promoción

9.3 Tabla transactions

La tabla transactions representa el historial de compras del usuario. Para el MVP se usó un usuario demo, pero el diseño permite conectar múltiples usuarios en una versión posterior.

Campo	Tipo	Descripción
id	uuid	Identificador único de transacción
user_id	text	Identificador del usuario
category	text	Categoría de compra
amount	numeric	Monto de la transacción
created_at	timestamp	Fecha de registro

10. Motor de Recomendación Contextual

El motor de recomendación es la parte que se presenta como inteligencia aplicada del proyecto. No se implementó un modelo generativo ni un modelo predictivo entrenado, porque el alcance del hackathon no requería ni permitía entrenar modelos con grandes cantidades de datos. En su lugar, se creó un sistema de recomendación contextual basado en reglas y ponderaciones claras.

10.1 Variables Analizadas

- Afinidad por categoría: se obtiene contando las categorías más frecuentes en el historial de transacciones.
- Proximidad: se calcula usando coordenadas de usuario y comercio.
- Promoción activa: se otorga mayor prioridad a comercios con beneficios activos.
- Contexto temporal: se clasifica el momento del día como mañana, tarde o noche para orientar futuras recomendaciones.

10.2 Cálculo del Score

El score final permite ordenar los comercios. Un comercio con categoría coincidente, menor distancia y mayor descuento obtiene una mejor posición en el feed. Esto ayuda a que el usuario vea primero lo que probablemente sea más relevante.

`score final = afinidad de categoría + proximidad + beneficio activo + contexto`

Un ejemplo práctico sería Burger Factory. Si el usuario ha comprado hamburguesas varias veces, el comercio pertenece a la categoría Hamburguesas, está cerca y tiene una promoción activa, el score resultante será alto. En la interfaz, esto se muestra como porcentaje de compatibilidad o afinidad.

11. Proximidad Geográfica y Cálculo de Distancia

La funcionalidad Nearby depende de la cercanía entre usuario y comercio. Para el MVP, se utilizó una ubicación simulada del usuario y coordenadas reales de prueba en los comercios. Técnicamente, esta estructura ya está preparada para recibir la ubicación GPS del teléfono en una versión móvil.

11.1 Fórmula Haversine

Para calcular distancia entre dos puntos geográficos se implementó una función basada en la fórmula Haversine. Esta fórmula estima la distancia entre dos coordenadas considerando la curvatura de la Tierra. En el endpoint `/api/nearby-businesses`, los comercios se ordenan de menor a mayor distancia.

Entrada: latitud usuario, longitud usuario, latitud comercio, longitud comercio
Salida: distancia aproximada en kilómetros

11.2 Integración Futura con GPS

En producción, la aplicación móvil enviaría al backend la ubicación actual del usuario. El backend reemplazaría la ubicación simulada por coordenadas reales y devolvería comercios cercanos ordenados por distancia. Esta decisión demuestra que el MVP no es una simulación aislada, sino una base técnica lista para escalar.

12. APIs Implementadas

Las APIs se diseñaron para que cada compañero del equipo consuma solamente lo que necesita. Esto evita que el frontend acceda directamente a toda la lógica de datos y permite mantener una arquitectura más ordenada.

Endpoint	Objetivo	Consumidor principal
<code>/api/recommendations</code>	Devuelve recomendaciones personalizadas con score	Frontend Home
<code>/api/nearby-moment</code>	Devuelve una notificación inteligente	Geolocalización/Notificaciones
<code>/api/promotions</code>	Devuelve promociones activas	Frontend Home
<code>/api/user-history</code>	Devuelve historial de compras	Dashboard
<code>/api/nearby-summary</code>	Devuelve métricas para pitch	Dashboard/Pitch
<code>/api/nearby-businesses</code>	Devuelve comercios cercanos con distancia	Geolocalización/Mapa

12.1 /api/recommendations

Este endpoint consulta transacciones, detecta categoría favorita, consulta comercios y promociones, calcula el score y devuelve una lista ordenada. Es el endpoint principal para el sidebar o feed de DeUna Nearby.

12.2 /api/nearby-businesses

Este endpoint consulta la RPC de Supabase `get_nearby_merchants_with_user_stats` y devuelve comercios con `distanceKm`. Cuando la respuesta incluye `source: supabase`, significa que el endpoint está usando la función real de Supabase y no solamente un motor local de respaldo.

12.3 /api/nearby-moment

Este endpoint devuelve un mensaje de notificación contextual. Su función es alimentar la experiencia de Nearby Moments, por ejemplo: “Vimos que últimamente te interesan las hamburguesas. Burger Factory está a 250 m y tiene un beneficio DeUna exclusivo”.

13. Integración con Frontend y Roles del Equipo

Para evitar conflictos, el equipo se dividió por ramas y responsabilidades. El backend se integró primero, luego los demás módulos consumen las APIs. Este enfoque facilita el trabajo colaborativo y reduce el riesgo de que un integrante modifique código que no le corresponde.

Persona	Rama	Responsabilidad	APIs principales
Persona 1	backend	Supabase, APIs, motor de recomendación y proximidad	Todas
Persona 2	frontend-home	Home móvil y cards DeUna Nearby	/api/recommendations, /api/promotions
Persona 3	geolocation-notifications	Nearby Moments, notificaciones y proximidad	/api/nearby-moment, /api/nearby-businesses
Persona 4	dashboard-pitch	Dashboard de impacto y pitch	/api/user-history, /api/nearby-summary

Cada integrante debe trabajar en su propia rama y traer los cambios del backend mediante git pull origin backend o desde main cuando el backend ya esté integrado. La regla principal es no modificar las APIs sin avisar, porque todos los módulos dependen de esas rutas.

14. Seguridad, Variables de Entorno y Buenas Prácticas

El proyecto usa variables de entorno para evitar escribir credenciales directamente en el código. En el archivo .env.local se almacenan la URL del proyecto Supabase y la publishable key. La secret key no debe compartirse ni subirse al repositorio, porque otorga permisos privilegiados.

```
NEXT_PUBLIC_SUPABASE_URL=https://proyecto.supabase.co
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxx
```

El uso de variables de entorno es una práctica fundamental para separar configuración sensible del código fuente. Además, se recomienda mantener .env.local fuera de GitHub mediante .gitignore.

14.1 Control de Versiones

Git y GitHub se utilizaron para organizar el trabajo colaborativo. La rama main representa la versión integrada, mientras que ramas como backend, frontend-home, geolocation-notifications y dashboard-pitch permiten trabajar por módulo. Esta estrategia ayuda a resolver conflictos y facilita revisar cambios antes de unirlos.

15. Pruebas y Validación

La validación se realizó probando cada endpoint en el navegador local. La meta era confirmar que cada ruta respondiera con JSON y que el endpoint de proximidad devolviera source: supabase cuando la RPC estuviera creada correctamente.

Prueba	Resultado esperado
GET /api/recommendations	Lista de comercios ordenada por score
GET /api/nearby-moment	Mensaje inteligente con title y message
GET /api/promotions	Promociones activas

GET /api/user-history	Historial de compras del usuario demo
GET /api/nearby-summary	Resumen para dashboard y pitch
GET /api/nearby-businesses	Comercios con distanceKm y source: supabase

También se corrigieron errores comunes durante la integración, como dependencias faltantes, rutas duplicadas, imports incorrectos, archivos no mergeados entre ramas y falta de RPC en Supabase. Estos ajustes forman parte normal del proceso de integración en proyectos colaborativos.

16. Resultados Esperados e Impacto

El resultado del MVP es una base funcional para mostrar recomendaciones en la home móvil de DeUna, activar notificaciones inteligentes y presentar métricas de impacto a comercios. Desde el punto de vista del usuario, la solución genera una razón adicional para abrir DeUna. Desde el punto de vista del comercio, aumenta la visibilidad. Desde el punto de vista de DeUna, incrementa la posibilidad de más transacciones y monetización.

Actor beneficiado	Valor entregado
Usuario	Descubre comercios cercanos y beneficios personalizados
Comercio	Aparece frente a clientes con afinidad y cercanía
DeUna	Aumenta frecuencia de uso, transacciones y oportunidades de monetización

El impacto más importante es que la aplicación deja de ser únicamente un medio de pago y se convierte en un canal inteligente de descubrimiento comercial. Esta transformación mejora la propuesta de valor del ecosistema DeUna.

17. Limitaciones y Trabajo Futuro

El MVP presenta limitaciones propias de un desarrollo de hackathon. La ubicación del usuario es simulada, el historial es reducido, las promociones son de prueba y no se implementó un sistema real de push notifications. Sin embargo, la arquitectura permite evolucionar rápidamente.

17.1 Mejoras Futuras

- Integrar GPS real del dispositivo móvil.
- Implementar notificaciones push con Firebase Cloud Messaging o Apple Push Notification Service.
- Usar modelos de machine learning para aprender patrones de consumo más complejos.
- Incorporar variables externas como clima, eventos o zonas de alto tráfico.
- Crear un panel real para que los comercios gestionen campañas y promociones.
- Integrar pagos reales dentro del flujo “Ver beneficio -> pagar con DeUna”.
- Aumentar la seguridad con políticas Row Level Security en Supabase.

18. Conclusiones

DeUna Nearby demuestra que es posible convertir una aplicación de pagos en una plataforma de descubrimiento comercial inteligente mediante una integración relativamente ligera. El MVP aprovecha datos transaccionales, ubicación y promociones para generar recomendaciones contextualizadas, lo cual puede aumentar el uso de la aplicación y beneficiar a comercios locales.

La solución destaca porque no propone una aplicación adicional, sino una funcionalidad integrada a DeUna. Esto reduce fricción para el usuario y facilita su adopción. Además, el backend queda preparado para ser consumido por diferentes módulos del equipo, manteniendo una arquitectura clara y escalable.

Finalmente, el proyecto es defendible técnicamente porque incluye Supabase real, una RPC, endpoints Next.js, cálculo de distancia y un motor de recomendación contextual. Aunque no se implementó IA generativa ni un modelo entrenado, sí se implementó inteligencia aplicada a datos, lo cual responde al objetivo del reto y permite construir una demo convincente.

Referencias

American Psychological Association. (2025). Headings. APA Style.

<https://apastyle.apa.org/style-grammar-guidelines/paper-format/headings>

Next.js. (2026). Route Handlers. <https://nextjs.org/docs/app/getting-started/route-handlers>

Next.js. (2026). File-system conventions: route.js. <https://nextjs.org/docs/app/api-reference/file-conventions/route>

PostgreSQL Global Development Group. (2026). CREATE FUNCTION. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/sql-createfunction.html>

Supabase. (2026). JavaScript: Initializing.

<https://supabase.com/docs/reference/javascript/initializing>

Supabase. (2026). JavaScript API reference. <https://supabase.com/docs/reference/javascript>

Apéndice A. Ejemplo de Esquema SQL

```
create table if not exists merchants (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  category text,
  description text,
  latitude double precision,
  longitude double precision,
  active boolean default true
);

create table if not exists promotions (
  id uuid primary key default gen_random_uuid(),
  merchant_id uuid references merchants(id),
  title text,
  description text,
  discount integer,
  active boolean default true
);

create table if not exists transactions (
  id uuid primary key default gen_random_uuid(),
  user_id text,
  category text,
  amount numeric,
  created_at timestamp default now()
);
```

Apéndice B. RPC de Supabase

```

create or replace function get_nearby_merchants_with_user_stats(
  user_lat double precision,
  user_lng double precision
)
returns table(
  id uuid,
  name text,
  category text,
  description text,
  latitude double precision,
  longitude double precision
)
language sql
as $$
  select
    m.id,
    m.name,
    m.category,
    m.description,
    m.latitude,
    m.longitude
  from merchants m
  where m.active = true;
$$;

```

Apéndice C. Endpoints Para Verificación

```

http://localhost:3000/api/recommendations
http://localhost:3000/api/nearby-moment
http://localhost:3000/api/promotions
http://localhost:3000/api/user-history
http://localhost:3000/api/nearby-summary
http://localhost:3000/api/nearby-businesses

```

Para considerar el backend conectado a Supabase, el endpoint `/api/nearby-businesses` debe responder con `source: supabase`. Si responde `source: engine` o se genera un error, significa que la RPC o las tablas todavía no están disponibles en la base de datos.